PawStay

By
Aryan Yadav
Larry Page
150096725023

Case study no.29

Index

1. Introduction to the Case Study
2. Problem Statement / Case Background (Abstract)
3. Problem Statement / Case Study Design
4. Methods & Algorithms / Technology Applied in the Problem Statement
5. Problem Statement / Case Study Implementation Details and Snapshots
6. Problem Statement / Case Study Results and Conclusion
7. References

1. Introduction to the Case Study

PawStay is a full-stack frontend web application designed to bridge the gap between pet owners and verified local pet sitters and dog walkers. In an era where pet ownership is at an all-time high, the demand for reliable, transparent, and convenient pet care services has never been greater. This case study documents the design, development, and implementation of PawStay — a responsive web platform built using core web technologies: HTML5, CSS3, and Vanilla JavaScript.

The platform addresses a real-world business problem faced by millions of pet owners: finding trustworthy, background-checked care providers on short notice. PawStay solves this by providing a structured marketplace where sitter profiles, real-time availability calendars, transparent pricing calculators, and booking management are all unified within a single, intuitive interface.

1.1 Project Overview

Attribute	Detail
Project Name	PawStay — Pet Sitting Network
Industry	Pet Care / Service Marketplace
Platform	Responsive Web Application
Technologies	HTML5, CSS3, Vanilla JavaScript
Total Files	4 core files (2 HTML, 1 CSS, 1 JS)
Target Users	Pet owners seeking local sitters & walkers

1.2 Motivation

Pet care is a $150+ billion global industry. Despite its size, most pet owners rely on informal networks (word of mouth, social media) to find sitters — lacking transparency in vetting, pricing, and availability. PawStay aims to formalize this experience with a professional, accessible digital platform.

2. Problem Statement / Case Background (Abstract)

2.1 Problem Statement

Connect pet owners with verified local sitters and walkers. The core challenge is multi-dimensional:

- Discovery: Pet owners lack a centralized, filtered interface to find qualified sitters by pet type, experience level, and date availability.
- Trust: There is no standard mechanism for verifying sitter credentials and displaying social proof (ratings, reviews).
- Pricing Clarity: Service pricing is opaque — especially with surcharges (holidays) and discounts (multi-pet, bundles).
- Booking Flow: Transitioning from browsing a profile to confirming a booking is fragmented and friction-heavy.

2.2 Abstract

This case study presents the design and implementation of PawStay, a responsive web application that resolves pet care discovery and booking challenges. The system features a dual-page architecture: sitters.html (profile browsing and filtering) and Availability.html (date-based booking with live price calculation). A shared stylesheet (styles.css) ensures visual consistency across both pages, while a single JavaScript file (app.js) orchestrates all dynamic behavior including sitter card rendering, calendar generation, price computation, modal management, and local-storage-based booking persistence.

Key engineering decisions include: a data-driven sitter card renderer, a deterministic availability generator, a composable price calculator algorithm supporting holiday surcharges and multi-level discounts, and a fully accessible UI with ARIA roles and keyboard navigation support.

2.3 Scope

Dimension	In Scope	Out of Scope
Authentication	Local booking state (localStorage)	Server-side login / OAuth
Payments	Price calculation display	Payment gateway integration
Backend	Static data in JS constants	Database / REST API
Notifications	Toast messages (UI)	Email / SMS notifications
Maps	Location text display	Google Maps integration

3. Problem Statement / Case Study Design

3.1 System Architecture Diagram

PawStay follows a client-side Single Page Application (SPA)-like architecture with two HTML entry points and shared static assets:

System Architecture

```
+-----------------------------------------------------+
|              PawStay Web Application                |
+-----------------------------------------------------+
|                                                     |
|   +------------------+  +--------------------+  |
|   |  sitters.html    |  | Availability.html  |  |
|   | (Profile Grid)   |  | (Booking Calendar) |  |
|   +--------+---------+  +----------+---------+  |
|            |                       |             |
|            +----------+------------+             |
|                       |                         |
|              +--------v--------+                 |
|              |    app.js       |                 |
|              | (Shared Logic)  |                 |
|              +--------+--------+                 |
|                       |                         |
|     +-----------------+-------------------+      |
|     |                 |                   |      |
| [SITTER DATA]  [AVAIL DATA]  [BOOKINGS]  |      |
| (SITTERS[])  (buildAvail())  (localStorage)      |
|                                                     |
|         styles.css (Shared Stylesheet)              |
+-----------------------------------------------------+
```

3.2 Application Flow Diagram

User Journey / Application Flow

```
[User Lands on sitters.html]
         |
         v
[Filter Bar: Pet Type / Experience / Date]
         |
         v
[Sitter Cards Rendered (renderSitters)]
         |
  +-------+--------+
  |                |
  v                v
[Contact Modal] [Availability.html?sitter=ID]
  |                |
  v                v
[Send Message]  [Select Sitter Tab]
```

```
                    |
                    v
              [Calendar Renders]
                    |
              [Click Available Date]
                    |
              [Booking Modal Opens]
              [Service / Pets / Duration]
              [Live Price Calculator]
                    |
              [Confirm Booking]
                    |
        +------------+------------+
        |            |            |
  [localStorage]  [Calendar  [Toast
   saveBooking]   re-render]  notification]
        |
  [My Bookings Section renders]
```

3.3 Data Model Design

The SITTERS array acts as an in-memory database. Each sitter object contains all attributes needed to render cards and drive the booking flow:

Field	Type	Purpose
id	Number	Unique identifier, used for URL params & tab selection
name / initials	String	Display name and avatar fallback initials
color	HEX String	Personalized avatar background color
rating / reviews	Number	Star rendering and social proof display
rate	Number (INR)	Base hourly rate for price calculation
pets	String[]	Accepted pet types (Dog, Cat, Bird, Rabbit)
fencedYard	Boolean	Yard status badge on profile card
experience	Enum String	Filter dimension: Beginner / Intermediate / Expert
services	String[]	Service chips shown on card (Walk, Sit, Feed)
location	String	City display on profile

3.4 UI Component Design

The UI is composed of reusable, self-contained components — each rendered dynamically from the data model:

Component	Description
Holiday Banner	Full-width green banner alerting users to holiday surcharge dates
Navbar	Sticky, glassmorphic nav with desktop links + hamburger mobile menu
Hero Section	Gradient hero with stats (2,400+ sitters, 98% happy, 50k+ bookings)
Filter Bar	Floating card with pet, experience, date, and location filters
Sitter Card	Profile card with avatar, ratings, pet badges, service chips, pricing tags
Sitter Tabs	Scrollable pill tabs for selecting sitter on the availability page
Calendar	Month-grid with color-coded availability and holiday star markers
Booking Modal	Service/pet/duration selector with live price breakdown
Contact Modal	Form to message a sitter directly from their card
Bookings Grid	Confirmed bookings rendered from localStorage with cancel option
Toast System	Slide-in notifications for booking confirmations and cancellations

4. Methods & Algorithms / Technology Applied

4.1 Core Technologies

Technology	Role in PawStay
HTML5	Semantic markup: <article>, <aside>, <section>, <nav>, <header>, <footer>
CSS3	Custom properties (variables), Grid, Flexbox, clamp(), gradients, transitions
Vanilla JavaScript (ES6+)	DOM manipulation, event handling, data rendering, algorithms
localStorage API	Client-side booking persistence without a backend
Font Awesome 6.5	Icon library (CDN) for UI affordances
Google Fonts	Nunito (headings) + DM Sans (body text)
ARIA / WAI-ARIA	Accessibility roles, aria-labels, aria-live regions

4.2 Price Calculation Algorithm

The calcPrice() function implements a composable, rule-based pricing engine:

Price Calculation Algorithm

```
INPUT: { rate, duration, petCount, service, dateStr }

Step 1: Base Price
  base = rate * durationMultiplier
  (1hr=1x, 2hr=2x, half=4x, full=8x)

Step 2: Holiday Surcharge
  IF date IN [Dec24, Dec25, Dec31, Jan1]:
    holidaySurcharge = base * 0.25
  afterSurcharge = base + holidaySurcharge

Step 3: Multi-Pet Discount
  extraPets = max(0, petCount - 1)
  multiPetDiscount = afterSurcharge * (extraPets * 0.10)

Step 4: Bundle Discount (if service == "bundle")
  bundleDiscount = (afterSurcharge - multiPetDiscount) * 0.15

Step 5: Final Total
  total = afterSurcharge - multiPetDiscount - bundleDiscount
```

```
OUTPUT: { base, holidaySurcharge, multiPetDiscount,
          bundleDiscount, total, holiday }
```

4.3 Availability Generation Algorithm

The buildAvailability() function generates a deterministic, pseudo-random calendar state for two months:

Availability Data Algorithm

```
buildAvailability():
  For each day in [current month, next month]:
    IF day is in the past: SKIP
    ELSE:
      mod = (day * 7 + month * 13) % 10
      IF mod < 5 : status = "available"  (50%)
      IF mod < 8 : status = "limited"    (30%)
      ELSE       : status = "booked"     (20%)

  Override: today+1, today+2 = "available"
            today+3          = "limited"

Holiday Detection:
  isHoliday(date):
    return date IN [[11,24],[11,25],[11,31],[0,1]]
```

4.4 CSS Design System

The stylesheet is built on a comprehensive CSS custom properties system (design tokens):

Token Category	Variables
Brand Colors	--primary (#E07A5F), --secondary (#81B29A), --accent (#F2CC8F)
Semantic Colors	--bg, --card-bg, --text, --text-light, --text-muted, --border
Shadows	--shadow-sm, --shadow-md, --shadow-lg
Borders	--radius-card (16px), --radius-btn (8px), --radius-pill (999px)
Typography	--font-head (Nunito), --font-body (DM Sans)
Animation	--transition (.22s cubic-bezier easing)
Calendar Colors	--cal-green, --cal-red, --cal-yellow, --cal-grey

4.5 Responsive Design Breakpoints

Breakpoint	Target	Changes Applied
1024px and below	Tablets	Sitter grid → 2 cols; calendar sidebar stacks; bookings → 2 cols
768px and below	Mobile	Hamburger menu; all grids → 1 col; filter groups go full-width
480px and below	Small phones	Modal buttons stack vertically; toast repositions to bottom-left

4.6 Accessibility Implementation

PawStay implements WCAG-aligned accessibility features throughout:

- Semantic HTML: <article>, <aside>, <nav>, <main>, <header>, <footer> landmarks
- ARIA roles: role="list", role="grid", role="button", role="tab", role="dialog"
- aria-label on all interactive elements (buttons, links, inputs)
- aria-live="polite" on calendar title and price breakdown for dynamic updates
- aria-current="page" on active navigation links
- Keyboard navigation: Enter/Space triggers calendar date selection
- Focus management: body scroll lock when modals open, Escape to dismiss

5. Implementation Details and Snapshots

5.1 File Structure

Project File Structure

```
pawstay/
├── sitters.html          # Page 1: Sitter profile browsing & filtering
│      ├── Holiday banner
│      ├── Navbar (desktop + mobile)
│      ├── Hero section with stats
│      ├── Filter bar (pet, exp, date, location)
│      ├── Sitter cards grid (injected by JS)
│      ├── How It Works section
│      ├── Pricing information section
│      ├── My Bookings section
│      └── Contact modal
│
├── Availability.html     # Page 2: Calendar booking & price calculator
│      ├── Holiday banner
│      ├── Navbar
│      ├── Availability hero
│      ├── Sitter selector tabs (6 sitters)
│      ├── Calendar grid (month view)
│      ├── Sidebar (sitter info + pricing rules + tips)
│      ├── My Bookings section
│      └── Booking confirmation modal
│
├── styles.css            # Shared design system (~1,400 lines)
│      ├── CSS custom properties (design tokens)
│      ├── Reset & base styles
│      ├── Navbar, hero, filter bar, sitter card, calendar
│      └── Modals, toasts, bookings, responsive breakpoints
│
└── app.js                # Shared JavaScript (~600 lines)
       ├── Section 1:  SITTERS[] data array (6 sitters)
       ├── Section 2:  buildAvailability() algorithm
       ├── Section 3:  Utility functions (starsHTML, formatDate, etc.)
       ├── Section 4:  calcPrice() pricing engine
       ├── Section 5:  initContactModal()
       ├── Section 6:  renderSitterCard() + initFilters()
       ├── Section 7:  initCalendar() + renderCalendar()
       ├── Section 8:  initBookingModal()
       ├── Section 9:  initSitterTabs() + updateSidebar()
       ├── Section 10: initHamburger()
       ├── Section 11: renderBookings() (localStorage)
       └── Section 12: DOMContentLoaded init()
```

5.2 sitters.html — Page 1 Snapshot Description

The sitters page is the primary landing page. It renders dynamically from JavaScript data:

- Holiday Banner: A full-width green gradient banner at the very top warns users of +25% surcharge dates (Dec 24, 25, 31 & Jan 1).
- Sticky Navbar: Glassmorphic (backdrop-filter: blur) navigation with PawStay logo, page links, booking badge, and a prominent "Book Now" CTA.
- Hero Section: Gradient background with large heading, descriptive subtext, and four platform statistics: 2,400+ Verified Sitters, 98% Happy Pet Parents, 50k+ Bookings, 4.9★ Average Rating.
- Filter Bar: Floats over the hero with filters for Location, Pet Type (Dog/Cat/Bird/Rabbit), Experience Level, and date range. Applies in real-time on change.
- Sitter Cards Grid: 3-column responsive grid (2 on tablet, 1 on mobile). Each card shows: colored avatar with initials, star ratings, hourly rate in INR, accepted pet badges, experience level badge, fenced yard status, service chips, pricing discount tags, and action buttons (Availability, Contact).

5.3 Sitter Profile Cards — Key Fields

Sitter	Rate (₹/hr)	Key Features
Priya Mehta (4.8★)	₹1,499	Expert · Dog, Cat · Fenced Yard · 127 reviews
Jake Rivera (4.6★)	₹1,249	Intermediate · Dog, Cat, Bird · 89 reviews
Ananya Iyer (5.0★)	₹1,849	Expert · Dog, Cat, Rabbit · Fenced Yard · 214 reviews
Tom Nguyen (4.2★)	₹999	Beginner · Cat, Bird · Budget option · 43 reviews
Sara Okafor (4.9★)	₹1,699	Expert · All pets · Fenced Yard · 176 reviews
Liam Foster (4.4★)	₹1,199	Intermediate · Dog, Rabbit · Fenced Yard · 61 reviews

5.4 Availability.html — Page 2 Snapshot Description

The availability page provides the core booking interface:

- Sitter Selector Tabs: Horizontally scrollable pill buttons, one per sitter. Each tab shows a colored avatar and name. Selecting a tab updates the sidebar info and calendar state. The sitter ID is also accepted via URL parameter (?sitter=3) from the profile card link.
- Month Calendar Grid: 7-column CSS Grid aligned to weekday headers (Sun–Sat). Each date cell is color-coded: green (available), yellow (limited), red (booked), grey (past). Holiday dates show a ⭐ star marker. Today is highlighted with a primary-color border.
- Sidebar: Three cards showing: (1) selected sitter details (name, rate, experience, accepted pets), (2) pricing rules quick reference, (3) booking tips.
- Booking Modal: Opens on clicking an available/limited date. Displays: formatted date, sitter name, holiday notice (if applicable), service type dropdown, pet count input (1-5), duration selector, and a live-updating price breakdown box.

5.5 My Bookings Feature

Confirmed bookings are persisted in localStorage under the key "pawstay_bookings" and rendered in a card grid on both pages:

- Each booking card shows: sitter avatar and name, booking date, service type icon, pet count, duration, holiday flag (if applicable), and total price paid.
- A Cancel Booking button removes the booking from localStorage and re-renders the grid.
- The navbar badge (My Bookings 🔵) shows a live count of confirmed bookings.

6. Results and Conclusion

6.1 Features Implemented

Feature	Status
Sitter profile cards with ratings, pet badges, experience badges	Completed
CSS star rating system (full, half, empty stars)	Completed
Filter bar: pet type, experience level, date range	Completed
Date availability picker (month calendar)	Completed
Holiday surcharge detection (+25%)	Completed
Multi-pet discount (-10% per extra pet)	Completed
Walk + Feed bundle discount (-15%)	Completed
Live price calculator in booking modal	Completed
Sitter selector tabs with URL parameter support	Completed
My Bookings with localStorage persistence	Completed
Contact modal with form validation	Completed
Toast notification system	Completed
Responsive design (mobile, tablet, desktop)	Completed
Accessibility (ARIA, keyboard navigation)	Completed

6.2 Product Building Outcomes

The three core product dimensions from the problem statement were fully realized:

- Pet types accepted: Each sitter lists their accepted animals (Dog, Cat, Bird, Rabbit) as colored badges. The filter bar lets owners instantly narrow results by pet type.
- Fenced yard status: Displayed as a badge on every sitter card — green "Fenced Yard" or grey "No Yard" — allowing owners of active dogs or small animals to filter appropriately.
- Experience level: Three tiers (Beginner, Intermediate, Expert) are displayed as colored badges and available as a filter dimension, enabling owners to match their needs to sitter capability.

6.3 Pricing Strategy Outcomes

- Holiday surcharges: Automatically detected and applied to Dec 24, 25, 31 and Jan 1 bookings. The surcharge is surfaced in both the booking modal price breakdown and a site-wide banner.

- 2nd pet discount: The calcPrice() algorithm applies -10% per additional pet (up to -40% for 5 pets), shown as a line item in the price breakdown.
- "Walk + Feed" bundles: Selecting the bundle service type applies an additional -15% flat discount, encouraging higher-value service combinations.

6.4 Technical Achievements

- Zero dependencies: The entire application runs on pure HTML, CSS, and Vanilla JS — no frameworks or build tools required.
- Shared JS architecture: A single app.js file serves both pages using getElementById guards, reducing code duplication significantly.
- Design system: All colors, spacing, shadows, and fonts are managed through CSS custom properties, making the UI easily themeable.
- Performance: No blocking scripts; JS is deferred via DOMContentLoaded; CSS animations use transform and opacity (GPU-accelerated).

6.5 Conclusion

PawStay successfully demonstrates how a professional, feature-rich web application can be built using only foundational web technologies. The project covers the full product lifecycle from problem identification through UI design, algorithm implementation, and user experience polish. The pricing engine is mathematically sound, the calendar system handles edge cases (past dates, holidays, month boundaries), and the UI maintains accessibility standards throughout.

The application proves that separation of concerns (HTML for structure, CSS for presentation, JS for behavior) — when applied rigorously — produces maintainable, scalable code. The shared-file architecture, design token system, and modular JavaScript sections make PawStay an excellent template for service marketplace applications in any domain.

6.6 Future Enhancements

- Backend integration: Replace localStorage with a REST API (Node.js/Express + MongoDB) for persistent, server-side bookings.
- User authentication: JWT-based login for pet owners and sitters with separate dashboards.
- Real-time availability: WebSocket-based live calendar updates when multiple users book simultaneously.
- Map integration: Google Maps API to show sitter locations and calculate proximity.
- Payment gateway: Razorpay / Stripe integration for actual payment processing.
- PWA support: Service Worker and Web App Manifest for offline access and installability.

7. References

8. MDN Web Docs — HTML5 Semantic Elements.
https://developer.mozilla.org/en-US/docs/Web/HTML/Element

9. MDN Web Docs — CSS Custom Properties (Variables).
https://developer.mozilla.org/en-US/docs/Web/CSS/Using_CSS_custom_properties

10. MDN Web Docs — CSS Grid Layout.
https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_grid_layout

11. MDN Web Docs — localStorage API.
https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage

12. W3C — Web Content Accessibility Guidelines (WCAG) 2.1. https://www.w3.org/TR/WCAG21/

13. WAI-ARIA Authoring Practices 1.1 — Dialog (Modal).
https://www.w3.org/TR/wai-aria-practices-1.1/#dialog_modal

14. Font Awesome 6 Documentation. https://fontawesome.com/docs/web/

15. Google Fonts — Nunito & DM Sans. https://fonts.google.com/

16. CSS Tricks — A Complete Guide to Flexbox.
https://css-tricks.com/snippets/css/a-guide-to-flexbox/

17. JavaScript.info — LocalStorage and sessionStorage. https://javascript.info/localstorage